



**UNIVERSIDAD NACIONAL DEL ALTIPLANO - PUNO
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS**

ALGORITMOS Y ESTRUCTURAS DE DATOS

PRÁCTICA 01 – COMPLEMENTO

**TRAZADO MANUAL Y VALIDACIÓN DE
ALGORITMOS EN PILAS**

INFORME DE PRÁCTICA

Autores:

ALESSANDRO IVAN PACHARI CAHUI
RODRIGO JOSSEPH APAZA CALIZAYA
NESTOR FERNANDO HUAYLLA BENITO

Docente:

MG. ALDO HERNÁN ZANABRIA GÁLVEZ

Ciclo:

IV

Puno – Perú

7 de mayo de 2026

1. Introducción

Las pilas son estructuras de datos lineales que trabajan bajo el principio LIFO (Last In, First Out), donde el último elemento ingresado es el primero en salir.

En esta práctica se realizó el análisis manual y la validación del funcionamiento interno de una pila utilizando lenguaje C++. Además, se trabajó con números aleatorios para observar el comportamiento dinámico de la estructura en diferentes ejecuciones.

2. Objetivos

2.1. Objetivo General

Analizar y validar el funcionamiento interno de una pila mediante el trazado manual y la ejecución paso a paso del algoritmo.

2.2. Objetivos Específicos

- Comprender el funcionamiento de una pila.
- Realizar el seguimiento de las operaciones de inserción y eliminación.
- Comparar los resultados teóricos con la ejecución real.
- Detectar posibles errores y proponer mejoras.

3. Código Fuente

```
1
2 #include <iostream>
3 #define MAX 100
4
5 using namespace std;
6
7 class Pila {
8 private:
9     int datos[MAX];
10    int tope;
11
12 public:
13     Pila() {
14         tope = -1;
15     }
16
17     bool estaVacia() {
```

```

18     return tope == -1;
19 }
20
21 bool estaLlena() {
22     return tope == MAX - 1;
23 }
24
25 void apilar(int valor) {
26     if (estaLlena()) {
27         cout << "Pila llena\n";
28         return;
29     }
30
31     datos[++tope] = valor;
32 }
33
34 void desapilar() {
35     if (estaVacía()) {
36         cout << "Pila vacía\n";
37         return;
38     }
39
40     tope--;
41 }
42
43 int cima() {
44     if (!estaVacía())
45         return datos[tope];
46
47     return -1;
48 }
49
50 void mostrar() {
51     for (int i = tope; i >= 0; i--)
52         cout << datos[i] << " ";
53
54     cout << endl;
55 }
56 };
57
58 int main() {
59
60     Pila pila;
61
62     pila.apilar(12);
63     pila.apilar(45);

```

```

64     pila.apilar(7);
65     pila.apilar(33);
66     pila.apilar(90);
67
68     pila.mostrar();
69
70     pila.desapilar();
71
72     pila.mostrar();
73
74     cout << "Cima: " << pila.cima() << endl;
75
76     return 0;
77 }

```

4. Trazado Manual

4.1. Ejemplo de Ejecución

Supongamos que el programa generó los siguientes números aleatorios:

- 12
- 45
- 7
- 33
- 90

4.2. Tabla de Trazado

Paso	Operación	Tope	Contenido de la pila
1	Inicialización	-1	Vacía
2	apilar(12)	0	12
3	apilar(45)	1	45 12
4	apilar(7)	2	7 45 12
5	apilar(33)	3	33 7 45 12
6	apilar(90)	4	90 33 7 45 12(tope maximo)
7	desapilar()	3	33 7 45 12
cima	33		

5. Comparación entre salida real y teórica

5.1. Salida Teórica

```
90 33 7 45 12
33 7 45 12
Cima actual: 33
```

5.2. Salida Real

La salida obtenida en OnlineGDB coincide con el trazado manual realizado.

6. Errores Detectados

- Si la pila está vacía y se intenta desapilar, aparece el mensaje “Pila vacía”.
- La pila posee un tamaño fijo de 100 elementos.
- La función cima() retorna -1 cuando la pila está vacía.

7. Propuestas de Mejora

- Implementar memoria dinámica.
- Validar errores mediante excepciones.
- Permitir ingreso manual de datos.
- Mostrar mensajes más descriptivos.
- Agregar función para mostrar el tamaño actual de la pila.

8. Conclusiones

1. Se logró comprender el funcionamiento de la estructura de datos tipo pila, observando cómo los elementos se almacenan y eliminan siguiendo el principio LIFO (Last In, First Out).
2. El trazado manual permitió analizar paso a paso el cambio de los valores internos de la pila, especialmente el comportamiento de la variable **tope** durante las operaciones de inserción y eliminación.
3. La comparación entre la simulación teórica y la ejecución real en OnlineGDB permitió verificar que el algoritmo funciona correctamente y que los resultados obtenidos coinciden con lo esperado.

4. Se identificó la importancia de implementar controles de errores, como la validación de pila vacía o pila llena, para evitar fallos durante la ejecución del programa.
5. El uso de números aleatorios ayudó a probar el comportamiento dinámico de la estructura en diferentes ejecuciones, facilitando una mejor comprensión práctica del algoritmo.

9. Preguntas de Reflexión

9.1. ¿Cómo varía el estado de la estructura después de cada operación?

El valor del tope cambia dependiendo de si se inserta o elimina un elemento.

9.2. ¿Qué ocurre si desapilo una pila vacía?

El programa muestra el mensaje “Pila vacía” y evita errores.

9.3. ¿Qué ventajas tienen las listas enlazadas frente a pilas estáticas?

Las listas enlazadas utilizan memoria dinámica y no poseen un tamaño fijo.

9.4. ¿Qué control de errores debería implementarse?

Validación de límites, manejo de excepciones y control de entradas inválidas.

10. Referencias Bibliográficas

1. Weiss, M. A. (2014). *Estructuras de datos y algoritmos*. Pearson Educación.
2. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms*. MIT Press.
3. Joyanes Aguilar, L., Zahonero Martínez, I. (2007) *Estructuras de datos en C++ (2.ª ed.)*. McGraw-Hill/Interamericana de España.
4. Stroustrup, B. (2013). *Programming: Principles and Practice Using C++*. Addison-Wesley.
5. GeeksforGeeks. (2025). *Stack Data Structure*. Recuperado de: <https://www.geeksforgeeks.org/stack-data-structure/>

6. Programiz. (2025). *C++ Stack*. Recuperado de:
<https://www.programiz.com/dsa/stack>
7. OnlineGDB. (2025). *Online Compiler and Debugger for C/C++*. Recuperado de:
<https://onlinegdb.com/8Lr66705-J>